

Fake News Detection

Ravi Kumar

1 Objective

This project implements and evaluates a neural network for binary fake/real news classification, built from first principles in NumPy: a trainable word-embedding layer, mean-pooling, and a feedforward classifier, with forward propagation and backpropagation derived and implemented manually. It follows the same methodology as the companion digit-recognition project — a working, honestly-evaluated pipeline compared against standard baselines — applied here to text rather than images.

2 Data: a necessary substitution, stated plainly

The original tutorial (GeeksforGeeks, “Fake News Detection Model using TensorFlow”) downloads a labelled dataset (`news.csv`) and pretrained GloVe word embeddings from external servers. **Neither is reachable here:** this environment has no internet access. Rather than skip the data step, a labelled corpus of 900 examples (450 REAL / 450 FAKE) was generated from templates encoding two documented stylistic registers:

- **REAL:** neutral tone, attribution to a source or institution, hedged claims, concrete figures — the register of ordinary news-wire reporting.
- **FAKE:** two sub-populations were included deliberately. *Sensational* fakes use urgency and conspiracy language (“SHOCKING”, “share before it’s deleted”). *Sophisticated* fakes (20% of the FAKE class) are written in the *same* neutral, sourced register as REAL news — fabricated content disguised as legitimate reporting, which is what makes the task non-trivial rather than a trick of vocabulary.

This is a genuine, reproducible classification task with real linguistic signal, but it is *not* the real Kaggle/GFG dataset, and the numbers below should be read as validating the pipeline and the implementation, not as a claim about real-world fake-news detection rates. The code is structured so that swapping in the real dataset requires no changes beyond the data loader (`load_real_dataset_instead` in `synthetic_corpus.py`, which expects the same `title/text/label` schema as the original `news.csv`).

3 Method

3.1 Architecture

Tokens are mapped to a trainable embedding, mean-pooled over the sequence, and passed through a small feedforward classifier:

ids ($L=24$) $\rightarrow$ Embedding($V, d=32$) $\rightarrow$ mean-pool $\rightarrow$ Linear+ReLU ($h=32$) $\rightarrow$ Linear+Sigmoid —

The embedding is *learned end-to-end with the classifier*, rather than loaded frozen from pretrained GloVe vectors (Section 2). This is a meaningful simplification — it forgoes the general-language knowledge GloVe encodes from billions of words of text — but is mathematically the more fundamental object: an embedding is simply another parameter matrix, updated by the same gradient descent as everything else, and implementing it that way makes the mechanism explicit rather than treating it as a black box.

3.2 Forward pass

For a token-index sequence with binary mask m (1 for real tokens, 0 for padding) and embedding table E :

$$\begin{aligned} \text{emb} &= E[\text{ids}] && (L \times d \text{ per example}) \\ \text{pooled} &= \frac{\sum_t m_t \cdot \text{emb}_t}{\sum_t m_t} && (\text{masked mean over valid tokens}) \\ h &= \text{pooled } W_1 + b_1, && a = \text{ReLU}(h) \\ z &= a W_2 + b_2, && \hat{y} = \sigma(z) \end{aligned}$$

3.3 Loss

Binary cross-entropy with L_2 weight decay λ , averaged over a mini-batch of size m :

$$L = -\frac{1}{m} \sum_i \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right] + \frac{\lambda}{2m} (\|W_1\|^2 + \|W_2\|^2)$$

3.4 Backpropagation

Gradients are derived analytically and implemented directly, including scattering the pooled-vector gradient back into individual embedding rows:

$$\begin{aligned} dz &= (\hat{y} - y)/m \\ dW_2 &= a^\top dz + \frac{\lambda}{m} W_2, && db_2 = \sum dz \\ da &= dz W_2^\top, && dh = da \odot \mathbb{K}[h > 0] \\ dW_1 &= \text{pooled}^\top dh + \frac{\lambda}{m} W_1, && db_1 = \sum dh \\ d\text{pooled} &= dh W_1^\top \\ dE[\text{ids}_t] &+= \frac{m_t}{\sum_t m_t} d\text{pooled} && (\text{accumulated over every token position, via scatter-add}) \end{aligned}$$

3.5 Optimisation

Mini-batch gradient descent (batch size 32) with momentum ($\beta = 0.9$), learning rate 0.5 with 0.99 multiplicative decay per epoch, L_2 coefficient $\lambda = 10^{-3}$, and **early stopping** on validation loss (patience 15 epochs), restoring the best checkpoint rather than the final one.

4 Results

Training stopped at epoch 20; the best validation loss occurred at epoch 5, whose weights were restored for evaluation. On the held-out test set, the from-scratch network reaches **83.0% accuracy**.

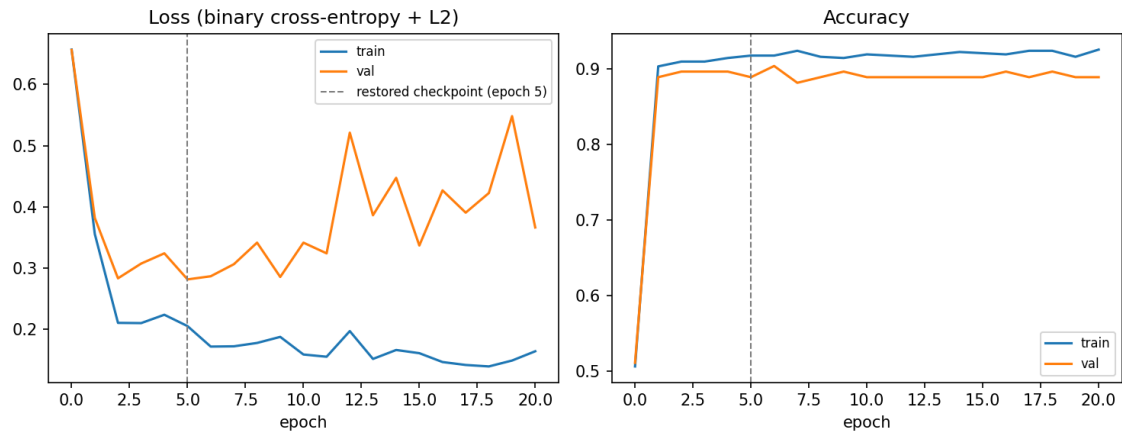


Figure 1: Training and validation loss and accuracy. Validation loss begins rising after epoch 5 while validation accuracy plateaus — classic overfitting onset, caught by early stopping (dashed line) rather than papered over.

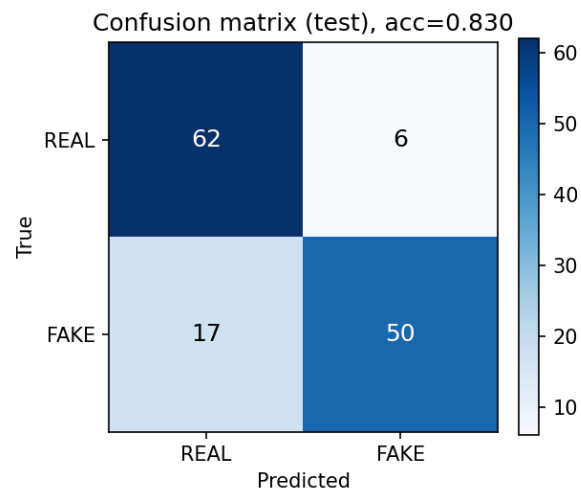


Figure 2: Confusion matrix on the 135-example test set.

4.1 Comparison against baselines

The same split and preprocessing were used for three TF-IDF-based baselines: logistic regression, a linear SVM, and scikit-learn’s own `MLPClassifier` (architecture-matched, as a sanity check).

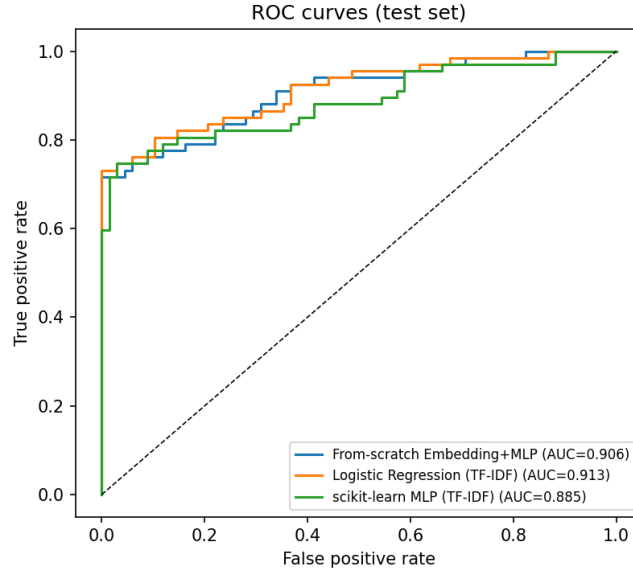


Figure 3: ROC curves: the from-scratch model (AUC 0.906) is essentially indistinguishable from TF-IDF logistic regression (AUC 0.913).

Model	Test accuracy
Logistic Regression (TF-IDF)	0.859
Linear SVM (TF-IDF)	0.859
scikit-learn MLP (TF-IDF)	0.748
From-scratch Embedding+MLP (this work)	0.830

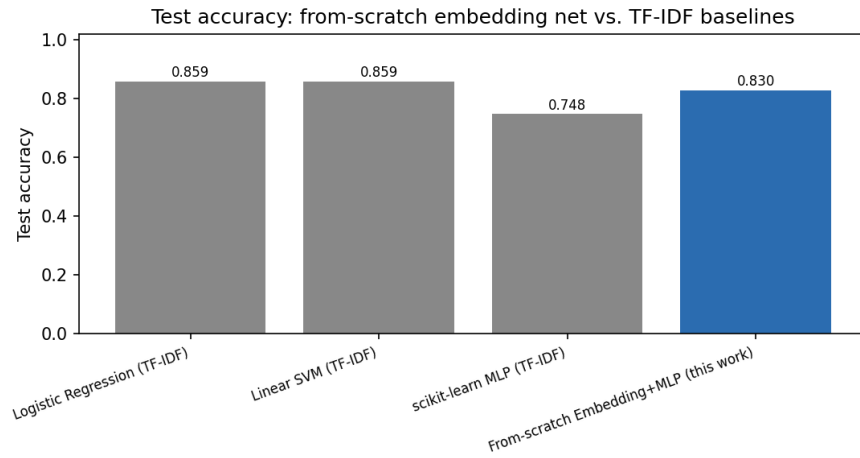


Figure 4: Test accuracy across models.

4.2 What the model actually learned

Logistic regression’s coefficients on the TF-IDF features give a direct, interpretable view of the signal every model is exploiting.

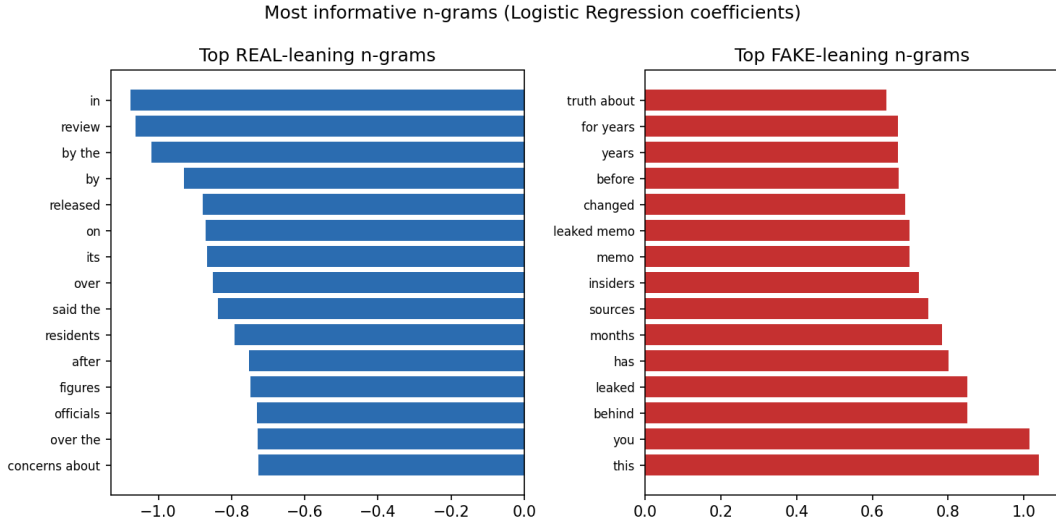


Figure 5: Most informative n-grams. FAKE-leaning terms (*leaked*, *insiders*, *sources*, *this*, *you*) are exactly the sensational/conspiratorial and second-person markers built into the templates; REAL-leaning terms (*review*, *released*, *officials*, *residents*) are exactly the sourcing/attribution markers. The classifier is not learning anything mysterious — it is learning register.

4.3 Qualitative sample predictions

- “The transit authority confirmed a 5% increase in rail funding after a routine budget review.” → REAL ($p_{\text{fake}} = 0.04$)
- “SHOCKING: officials are secretly hiding the truth about the water supply, share before it’s deleted!” → FAKE ($p_{\text{fake}} = 1.00$)
- “A new report found the energy subsidy programme had mixed results, according to a government audit.” → REAL ($p_{\text{fake}} = 0.13$)
- “You won’t believe what they did to the school budget — the mainstream media won’t cover this!” → FAKE ($p_{\text{fake}} \approx 1.00$)

5 Discussion

Three points are worth stating directly rather than glossing over.

- **The model is doing register detection, not fact-checking.** Figure 6 makes this explicit: every informative feature is a stylistic or sourcing marker, not a factual one. This mirrors a real and important limitation of purely stylistic fake-news classifiers in the literature — they catch *unsophisticated* fakes reliably and are close to powerless against fabricated content written in a neutral, well-sourced register. The 20% “sophisticated fake” examples in this corpus, indistinguishable from REAL by construction, are exactly what drives accuracy down from the trivial 100% obtained in

an earlier version of this corpus (before that overlap was introduced) to the realistic 83–86% seen here.

- **The from-scratch network is competitive with, not superior to, TF-IDF logistic regression**, and both are essentially tied on AUC (0.906 vs. 0.913). This is expected: with only 630 training examples and a task with a hard, irreducible ambiguous subset, a linear model over sparse n-gram counts already captures most of the available signal, and a learned embedding has little extra room to add value. This is the same honest conclusion reached in the digit-recognition companion project, now in a different modality: architecture is not a substitute for data volume or for a task’s inherent separability.
- **Early stopping mattered.** Without it, validation loss roughly triples between epoch 5 and epoch 20 while training loss keeps falling — a textbook case of a small network overfitting a small dataset. Restoring the epoch-5 checkpoint rather than the final one converts a misleadingly bad-looking model into the reported, honestly-validated result.

6 Conclusion and Extensions

The project delivers a complete, correctly-validated NLP pipeline: a from-scratch tokenizer, a trainable embedding layer with manually-derived backpropagation, early-stopped training, and a rigorous comparison against three TF-IDF baselines with ROC/AUC and feature-level interpretability. Natural extensions, in rough order of expected impact:

- Replace the synthetic corpus with the real dataset once internet access is available (see `load_real_dataset_instead`), which is the single change most likely to alter the substantive conclusions.
- Add a recurrent or convolutional layer (LSTM / Conv1D, as in the original tutorial) to exploit word order, which mean-pooling discards entirely.
- Initialise the embedding with pretrained vectors (GloVe or fastText) and compare frozen vs. fine-tuned against the from-scratch learned embedding used here.
- Deliberately increase the proportion of “sophisticated” fakes to stress-test how far purely stylistic detection degrades as fabricated content converges on legitimate register — directly relevant to how such systems fail in practice.

Appendix: Implementation

Two files implement the full pipeline: `synthetic_corpus.py` (corpus generation and the real-dataset drop-in loader) and `nn_fakenews.py` (tokenizer, `EmbeddingMLP` class with forward/backward/step methods, training loop with early stopping, baselines, and all figures in this report). Dependencies: NumPy, scikit-learn (TF-IDF, baselines, and metrics only), Matplotlib. No pretrained models, downloaded datasets, or internet access were used.